



UNIVERSIDADE PAULISTA – UNIP

CIÊNCIAS DA COMPUTAÇÃO

CRIPTOGRAFIA

Gabriel Henrique Lopes
Luis Henrique de Souza Nunes
Kauan da Silva Teodoro
Douglas Bernardo dos Santos
Patryck [sobrenome]

Trabalho apresentado à disciplina de **Criptografia**,
do curso de **Ciências da Computação**,
da Universidade Paulista – UNIP,
como parte dos requisitos para obtenção de nota parcial.

Professor: Nome do Professor

Sumário

1	Introdução	2
2	Problema abordado e objetivos	3
3	Aportes teóricos	4
4	Desenvolvimento:	5
5	Estrutura do sistema implantado	6
5.1	Observações sobre a implementação	6
6	Resultados	8
6.1	Resultados e testes	8
7	Considerações Finais	10
8	Apêndice — Código-fonte (implementação em Python)	12

1 Introdução

A segurança da informação é um dos pilares fundamentais da era digital, sendo indispensável para garantir a confidencialidade, integridade e autenticidade dos dados transmitidos por meio das redes de comunicação. Nesse contexto, a criptografia surge como uma ferramenta essencial, pois possibilita a proteção de informações sensíveis contra acessos não autorizados.

Entre as diversas técnicas criptográficas existentes, o algoritmo RSA destaca-se por sua robustez e ampla utilização em sistemas modernos de segurança. Desenvolvido em 1977 por Ronald Rivest, Adi Shamir e Leonard Adleman, o método baseia-se em princípios matemáticos da teoria dos números, utilizando a dificuldade de fatoração de grandes números primos como base para sua segurança.

O uso da criptografia RSA é recorrente em aplicações que exigem alto nível de proteção, como comunicações seguras na internet, assinaturas digitais, autenticação de usuários e proteção de dados confidenciais. Sua estrutura assimétrica, que utiliza um par de chaves — uma pública e outra privada —, permite que mensagens sejam cifradas por um emissor e decifradas apenas pelo destinatário autorizado, garantindo a segurança na troca de informações.

Neste trabalho, busca-se compreender o funcionamento do algoritmo RSA e sua importância no contexto da segurança digital, por meio da implementação prática de um programa capaz de cifrar e decifrar mensagens. A proposta envolve aplicar os conceitos teóricos da criptografia em um cenário que simula a comunicação segura entre agentes que atuam em uma área ambientalmente contaminada e de acesso restrito, assegurando que apenas pessoas autorizadas possam transmitir e receber informações.

Assim, pretende-se demonstrar a relevância da criptografia RSA não apenas como um recurso tecnológico, mas como um elemento essencial para a preservação da segurança da informação em ambientes críticos e de alto risco.

2 Problema abordado e objetivos

O avanço das tecnologias de comunicação trouxe inúmeros benefícios à sociedade, mas também ampliou os riscos relacionados à interceptação e uso indevido de informações sensíveis. Em cenários que envolvem dados críticos, como o controle de acesso a áreas contaminadas ou de risco ambiental, a necessidade de proteger a troca de informações entre agentes torna-se indispensável.

Dessa forma, o problema central abordado neste trabalho consiste em garantir a segurança da comunicação entre agentes de campo e as centrais de controle, evitando que mensagens sejam lidas ou alteradas por terceiros não autorizados.

O objetivo principal é desenvolver e aplicar uma solução criptográfica baseada no algoritmo RSA, que permita cifrar e decifrar mensagens trocadas entre as partes envolvidas. Além disso, busca-se compreender os fundamentos teóricos da criptografia, avaliar suas aplicações e discutir as vantagens e limitações da técnica escolhida.

3 Aportes teóricos

Criptografia: ciência que estuda métodos de codificação de mensagens, garantindo a confidencialidade e autenticidade das informações.

Criptografia assimétrica: técnica que utiliza um par de chaves distintas — uma pública e uma privada — para cifrar e decifrar mensagens.

Algoritmo RSA: desenvolvido em 1977 por Rivest, Shamir e Adleman, baseia-se na dificuldade de fatorar grandes números primos. É amplamente utilizado em comunicações seguras, autenticação e assinaturas digitais.

Segurança da informação: conjunto de práticas que visam proteger dados contra acessos indevidos, perdas e modificações.

4 Desenvolvimento:

Para o desenvolvimento do sistema, foram seguidas as seguintes etapas:

Pesquisa teórica sobre os fundamentos da criptografia e funcionamento do algoritmo RSA.

Definição dos requisitos do sistema, como capacidade de cifrar e decifrar mensagens de até 128 caracteres.

Implementação do algoritmo em linguagem de programação escolhida pelo grupo, aplicando operações matemáticas de geração de chaves, encriptação e deciptação.

Testes práticos, simulando a comunicação entre agentes e central de comando, validando a integridade das mensagens transmitidas.

5 Estrutura do sistema implantado

5.1 Observações sobre a implementação

O sistema foi desenvolvido em Python, de forma didática e sem o uso de bibliotecas externas, com o objetivo de demonstrar o funcionamento básico do algoritmo RSA. A implementação foi dividida em duas partes independentes, representando os dois lados do processo de comunicação segura: a criptografia (chave pública) e a descryptografia (chave privada).

Parte 1 — Criptografia (uso da chave pública)

O primeiro código é responsável apenas por realizar a criptografia da mensagem, utilizando a chave pública formada pelos valores e e n . Essa parte pode ser executada isoladamente, sem necessidade de conhecer os números primos usados na geração da chave privada.

A função principal é `criptografar(texto, e, n)`, que realiza as seguintes etapas:

- Converte o texto original em bytes utilizando o método `encode()`;
- Calcula o tamanho máximo de cada bloco com base no valor de n ;
- Divide a mensagem em blocos de bytes dentro desse limite;
- Converte cada bloco em número inteiro por meio de `int.from_bytes()`;
- Cifra cada bloco com a operação modular `pow(valor, e, n)`;
- Retorna duas listas: uma com os blocos cifrados (`cifra`) e outra com o tamanho de cada bloco original (`tamanho`).

Essa função representa a fase pública do RSA — qualquer pessoa que possua a chave (e, n) pode cifrar uma mensagem, mas não é capaz de decifrá-la sem o valor de d .

Parte 2 — Descryptografia (uso da chave privada)

O segundo código contém a geração dos parâmetros do algoritmo e a função responsável por reverter o processo. São definidos dois números primos p e q , a partir dos quais são calculados:

- O módulo $n = p * q$;
- O valor de $\varphi(n)$, obtido por $(p - 1) * (q - 1)$;
- O expoente público $e = 17$;

-
- O expoente privado d , calculado por meio de um laço `for` que encontra o inverso multiplicativo de e módulo $\varphi(n)$.

A função `descriptografar(cifra, d, n)` aplica a operação inversa da cifragem. Cada número cifrado é elevado a d módulo n (`pow(c, d, n)`), retornando o valor original em bytes, que é decodificado novamente para texto legível.

Fluxo geral de execução

A execução do sistema ocorre em duas etapas distintas:

1. O remetente executa o primeiro script, que cifra a mensagem usando apenas a chave pública (e, n);
2. O destinatário, em posse dos valores p, q, d e n , executa o segundo script para descriptografar a lista de números recebida.

Essa separação reproduz o modelo real de comunicação segura do RSA, no qual a parte pública é compartilhada livremente, enquanto a privada permanece restrita ao proprietário da chave.

6 Resultados

6.1 Resultados e testes

Após a implementação das duas partes do sistema RSA, uma dedicada à criptografia (pública) e outra à descriptografia (privada), foram realizados testes práticos para validar a comunicação completa entre os módulos.

A mensagem utilizada nos testes contém letras, números e símbolos especiais, garantindo a compatibilidade do algoritmo com diferentes tipos de caracteres.

Geração das chaves

No módulo de descriptografia, o programa calcula automaticamente os valores de n , $\varphi(n)$, e e e d , exibindo-os no console. **Exemplo (saída real do código):**

```
n = 3233, phi = 3120, chave pública = 17, chave privada = 2753
```

Esses valores são usados para validar o processo de cifragem e decifragem entre os dois scripts.

Cifragem da mensagem

No primeiro código, o usuário define uma mensagem de teste e chama a função `criptografar(msg, e, n)`. O resultado é uma lista de números inteiros que representam a mensagem cifrada com a chave pública.

Exemplo:

```
Criptografada: [3123, 1313, 2235, 1230, 1632, 2923, 1313, 2271, 1992, 1773, 1313, 1992, 884, 1313, 1230, 884, 1313, 1992, 612, 1632, 2412, 1632, 1992, 281, 2412, 3179, 612, 884, 2185, 2923, 2412, 1632, 1369, 3179, 1632, 678, 1992, 624, 2906, 538, 368, 529, 1802, 1962, 2653, 1794, 1175, 678, 1992, 1725, 501, 2825, 2502, 1097, 95, 485, 589]
```

Esses valores não possuem correspondência direta com caracteres legíveis, garantindo que o conteúdo original permaneça protegido.

Decifragem da mensagem

O segundo script utiliza a função `descriptografar(cifra, d, n)` para reverter o processo. Após a execução, o texto original é reconstruído:

```
Descriptografada: Mensagem de teste para criptografia, 0123456789, ;/.><~]
```

O resultado confirma que o processo foi realizado com sucesso e que a comunicação entre os dois módulos funciona corretamente.

Análise e comportamento

- A divisão em dois arquivos reflete o funcionamento real do RSA, separando a fase pública da privada;
- O código mostrou-se eficiente para mensagens curtas e médias;
- A codificação em UTF-8 garantiu compatibilidade com caracteres especiais e acentuação;
- Os valores primos utilizados ($p = 61$, $q = 53$) foram pequenos por questões didáticas, o que reduz a segurança, mas facilita o entendimento.

Em resumo, os resultados obtidos demonstram a eficácia do sistema em reproduzir todas as etapas do algoritmo RSA, desde a geração das chaves até a comunicação simulada entre emissor e receptor.

7 Considerações Finais

A nova estrutura modular do sistema permitiu representar de forma mais fiel o funcionamento prático da criptografia RSA, separando claramente as responsabilidades da chave pública e da chave privada.

A implementação demonstrou de maneira didática como o processo de cifragem pode ser realizado por qualquer pessoa que possua a chave pública (e , n), enquanto a recuperação da mensagem depende exclusivamente da chave privada (d , n), calculada a partir dos números primos p e q .

Mesmo utilizando valores pequenos e simplificados, o projeto atingiu seu objetivo educacional: mostrar o ciclo completo de uma comunicação segura da codificação à decodificação utilizando apenas operações matemáticas fundamentais.

Além disso, a divisão em dois módulos independentes melhora a legibilidade, a reutilização do código e a compreensão dos papéis de cada parte no processo criptográfico. Essa abordagem também facilita futuras expansões, como a inclusão de assinaturas digitais ou o uso de chaves de tamanho real para maior segurança.

Portanto, o sistema proposto cumpre integralmente sua função didática, ilustrando de forma prática e compreensível o princípio da criptografia assimétrica e a importância da proteção da informação no contexto computacional moderno.

Referências

- R. L. Rivest, A. Shamir, L. Adleman. *A Method for Obtaining Digital Signatures and Public-Key Cryptosystems*. 1978.
““
- Katz, J.; Lindell, Y. *Introduction to Modern Cryptography*. Capítulos sobre RSA.
- *Documentação oficial do Python*: funções `int.to_bytes`, `int.from_bytes` e `pow` (com parâmetro de módulo).
Disponível em: <https://docs.python.org/3/library/functions.html> ““

8 Apêndice — Código-fonte (implementação em Python)

Arquivo 1 — Criptografar.py (chave pública)

```
n = 3233
e = 17

def criptografar(texto, e, n):
    b = texto.encode()
    maxb = (n.bit_length() - 1) // 8
    blocos = []
    cifra = []
    tamanho = []
    for i in range(0, len(b), maxb):
        blocos.append(b[i : i + maxb])
    for ch in blocos:
        valor = int.from_bytes(ch, "big")
        cifra.append(pow(valor, e, n))
        tamanho.append(len(ch))
    return cifra, tamanho

msg = "Mensagem de teste para criptografia, 0123456789, ;/.><~]"
print("\nMensagem:", msg)
cifra, tamanho = criptografar(msg, e, n)
print("\nCriptografada:", cifra)
```

Arquivo 2 — Descriptografar.py (chave privada)

```
cifra = [ ... ] # valores obtidos na criptografia
```

```
p = 61
```

```
q = 53
```

```
n = p * q
```

```
phi_n = (p - 1) * (q - 1)
```

```
e = 17
```

```
d = None
```

```
for k in range(1, 10000):
```

```
    if (k * phi_n + 1) % e == 0:
```

```
        d = (k * phi_n + 1) // e
```

```
        break
```

```
def descriptografar(cifra, d, n):
```

```
    textodesc = bytearray()
```

```
    for c in cifra:
```

```
        m = pow(c, d, n)
```

```
        tamanho = (m.bit_length() + 7) // 8
```

```
        textodesc += m.to_bytes(tamanho, "big")
```

```
    textodesc = textodesc.decode()
```

```
    return textodesc
```

```
msg_desc = descriptografar(cifra, d, n)
```

```
print("\nMensagem descriptografada:", msg_desc)
```